

COMP1035 Software Engineering — Full Revision Notes

Complete lecture-by-lecture review (Lec 01–18)

English key points (verbatim definitions) + Chinese study aids

Contents

Course Map: Four Core SE Processes [Lec01]	4
1. Introduction to Software Engineering [Lec01]	5
Software vs “code”	5
What is software engineering?	5
2. Version Control & Git [Lec02/09/10]	6
Version control basics [Lec02]	6
Core Git operations	6
Branching workflow [Lec09]	6
Git platforms: GitLab / GitHub [Lec10]	7
3. Requirements Engineering [Lec03]	7
Why requirements matter	7
User vs System requirements	7
Four types of requirements	8
Stakeholders — three layers	8
Functional vs Non-functional requirements	8
Elicitation techniques + Ethnography	9

4. Requirements Elicitation: Documents [Lec04]	10
Use case diagram (as an elicitation tool)	10
Personas	10
Scenarios	10
User stories	11
5. Requirements Modelling: UML Diagrams [Lec05]	11
(1) Context Models	11
(2) Activity Diagrams	12
(3) Use Case Diagrams	12
(4) Sequence Diagrams	12
(5) Class Diagrams	13
(6) State Diagrams	13
How to choose a diagram?	14
6. Requirements Specification [Lec06]	14
7. Requirements Validation [Lec07]	14
8. Prototyping [Lec08]	15
9. Testing Methods & TDD [Lec11]	16
Testing methods	16
White-box coverage criteria	17
Black-box techniques	17
White-box vs Black-box (comparison)	17
Manual vs Automation testing	18
Test-Driven Development (TDD)	18
JUnit & metrics	19
10. Integration, System & Acceptance Testing [Lec12]	20
Integration testing	20
System testing	20
Acceptance testing (vs integration)	21
11. Configuration & Deployment [Lec13]	22
Why evolution	22
Configuration Management (CM) — four activities	22

Version control concepts	23
System building & CI	23
Change management	24
12. Evolution & Maintenance [Lec14]	24
Types of maintenance (verbatim)	25
Maintenance costs & prediction	25
Lehman's laws of program evolution (Lehman & Belady)	25
Reengineering & refactoring	26
13. Software Quality [Lec15]	27
QA team — three (four) things	27
Reviews & inspections	27
14. Agile Methods [Lec16]	28
History & motivation	28
Agile principles (the 12)	28
Frameworks (all uphold the Agile principles)	29
How to apply Agile to each stage	30
Scrum roles, artifacts & events	30
Obstacles / cons of Agile	30
15. Risk Management [Lec17]	31
Definitions (verbatim — learn exactly)	31
Identifying risks	31
Product risk reduction (3, by design)	31
Project risk: prioritisation & strategies	32
16. Project Planning [Lec18]	32
Project plan & scheduling	33
Planning diagrams	33
PERT & critical path (2025 Q3)	33
Gantt charts	33
Reviewing progress & Agile planning	33
17. Likely Exam Questions (by topic)	34
Requirements (Lec03–07)	34

Development & testing (Lec11–13)	35
Management (Lec14–18)	35
18. Worked Examples	36
A. Use case diagram: library system [Lec05]	36
B. Activity diagram: fork / join [Lec05]	36
C. Statement & condition coverage [Lec11]	36
D. PERT critical path [Lec18]	37
19. Exam-Day Quick-Recall Sheet	37
20. Glossary of Key Terms	39

How to use this — The **English** text is what you memorise and write in the exam (answers are in English). Definitions in “**quotation marks**” are taken **verbatim from the slides** — learn those word-for-word. Light-blue boxes are **中文助记**: short Chinese notes that only help you understand/remember, not what you write. Each part is tagged with its lecture, e.g. [Lec05].

Exam (Revision deck): text answers, **no code**; you may be asked to **draw diagrams**; some **multiple choice**; *write as much as you can*.

Course Map: Four Core SE Processes [Lec01]

The module follows the four **core SE processes** end to end:

- **Requirements & Specifications** — “Designing the system for what the customer wants.”
- **Development** — “Production of the software system.”
- **Validation & Testing** — “Checking that the software is what the customer wanted.”
- **Evolution & Maintenance** — “Changing code in response to new requirements.”

Two ideas run through everything: **all SE processes are designed to improve software quality** [Lec15], and **change is inevitable** [Lec14].

中文助记：全课就是这条流水线——需求/规格 -> 开发 -> 验证/测试 -> 演化/维护。把每一讲都挂到这条线上记。两条暗线：所有流程都为提质量、变化必然。

1. Introduction to Software Engineering [Lec01]

Software vs “code”

- **Software** is **more than code** — it is complex enough to “need a big process” to build. What you wrote before (small scripts that do one thing, not ready for real users, probably buggy) is a **component** of software, not software.
- Examples that need a managed process: a phone game with a global scoreboard, **Twitter**, **Google Chrome**, a government **Universal Credit** benefits system.

What is software engineering?

- **Software Engineering** — a **systematic, disciplined, engineering approach** to the **development, operation and maintenance** of software.
- **Software process** — the activities (and their order) that produce a software product. Phases: *Requirements & Specification -> Design (how to build) -> Implementation & Testing -> Evolution & Maintenance*.
- Why have a process at all? Historically software was often **poor quality, late and over budget**; since the 1970s/80s people kept inventing processes to better *engineer* software.

中文助记：“Software is complex enough to need a big process” 是关键句。写代码 ≠ 做软件：上学期写的是脚本/组件。SE = 用过程 / 方法 / 工具把“写代码”变成可管理、可度量、可重复的工程，解决软件质量差、超期、超预算的老问题。

2. Version Control & Git [Lec02/09/10]

Largely practical (labs); examined as **concept questions**.

Version control basics [Lec02]

- **Version Control (VCS)** — tracks the **multiple versions** of components; lets you step back to any previous commit and lets a team collaborate.
- **Repository** — stores all files plus the **full history of every commit**.
- The **three areas**: **working directory** (your edited files) -> **staging area (index)** (changes gathered for the next commit) -> **repository** (permanent commits).

Core Git operations

- **add** — move changes from working directory to the **staging area**.
- **commit** — create a **permanent snapshot** of the staged state (`git commit -m "msg";` -a stages all *tracked* files first).
- **push / pull** — send/receive commits to/from a remote (push can cause **merge conflicts** if local and remote diverge).
- **checkout** — restore files / switch branch / step back to a previous commit.
- **.gitignore** — file types Git ignores. **Tag** — a friendly name for a commit (e.g. a release version).

Branching workflow [Lec09]

- **Branch** — duplicate the work so it can be modified **in parallel**; later **merge** it back, which may create a **conflict** to resolve manually.
- Team flow: **feature branch** -> commit -> **merge / pull request** -> **review** -> merge into main. **Never develop directly on main** so it always stays usable.
- A **conflict** happens when two branches change the same lines; Git marks the conflicting region and you must edit it, then commit the resolution.

- Useful commands: **git status** (what's changed/staged), **git log** (commit history), **git clone** (copy a remote repo with its full history).

Git platforms: GitLab / GitHub [Lec10]

- Host repositories and add collaboration features:
 - **Issues** — building blocks for tracking work/bugs/tasks (centralized, transparent).
 - **Labels** — metadata to categorise/close issues. **Milestones** — group issues toward a target. **Merge / Pull Requests** — code review before merging. **CI/CD**.
- **Good commit messages**: concise summary (<=50 chars), be specific (“Fix login form validation bug”, not “Fix bug”), reference issues (Fix, Close, Resolve).

中文助记：三区——工作区 -> 暂存区 (**index**) -> 仓库。add->commit->push/pull, checkout 回退/切分支。分支流：feature branch->merge request->review->合并，别直接动 **main**。平台三件套：**Issues / Labels / Milestones** + 合并请求 + CI/CD。

3. Requirements Engineering [Lec03]

Why requirements matter

- **Incomplete requirements and lack of user involvement** top the list of causes of **project failure** (Standish CHAOS report). Understanding requirements, **not coding**, is the major problem.
- Brooks: *“the hardest single part of building a software system is deciding precisely what to build.”* As much as a **200:1 cost saving** comes from finding errors in the **requirements stage** vs the **maintenance stage**.

User vs System requirements

- **User requirements** — statements in **natural language + diagrams**; the services users expect and the operating constraints. *“What a user needs to be able to do.”*

- **System requirements (specifications)** — more detailed descriptions of functions, services and constraints; the **functional specification** “serves to precisely define what is to be implemented” and may be part of the **contract**. *“What the software must do to meet the user requirements.”*

Four types of requirements

1. **Business requirements** — the project’s purpose and the company’s expected gains.
2. **Stakeholder requirements** — the needs/expectations of stakeholders.
3. **Solution requirements** — technical: **functional** (system functionality) + **non-functional** (system qualities).
4. **Transition requirements** — steps to move the organisation from its current state to the desired state.

Stakeholders — three layers

- **Primary (direct users)** — hands-on daily users / departments that gain or lose; the “key stakeholders”.
- **Secondary (indirect users)** — people/groups affected by the project (e.g. local construction companies near a new park).
- **Tertiary (indirect)** — affected more indirectly (business owners, public, government agencies); often “external”, advisory/advocacy role.

Functional vs Non-functional requirements

- **Functional requirements** — “The services the system is expected to provide, how it should respond to specific inputs, and its behaviour in various situations.” May also state what the system should **not** do.
- **Non-functional requirements** — “Impose limitations on the services or functionalities offered by the system” (timing, development-process and standards constraints). Usually relate to the **system as a whole**, not individual features.
- Three NF sub-types: **Product** (e.g. throughput, number of concurrent users, uptime,

failure safety), **Organisational** (e.g. must use the university authentication service / IDs), **External** (e.g. must meet web-accessibility guidelines / legal standards).

- **Functional vs NF self-test** (slide example): “A user should be able to post a new image” = **functional**; “it should handle all major image types” / “users can post big high-definition images” = **non-functional** (constraints on the service).
- The **User-Req -> System-Spec** split: a user requirement (“a module convenor needs to check a student’s prerequisites”) expands into several precise system requirements (“shall be able to see a list of students”, “shall be able to see grades”, “shall be able to accept/reject”).

Elicitation techniques + Ethnography

- Techniques: interviews, questionnaires/surveys, observation, document analysis, prototyping. **Have a strategy** — one method helps interpret another.
- **Ethnography** — an **anthropological, observational** method; “*there’s no better way to learn than doing*” — the analyst gets involved in the client’s real work environment. **Focused ethnography** targets specific tasks/roles.
- **Four benefits of ethnography** (2024 Q3): (1) reveals **tacit/implicit knowledge**; (2) captures the **real process**, not the idealised one; (3) reveals **cooperation and coordination**; (4) **non-intrusive, low bias** (observe, don’t rely on what people say).

中文助记：需求最重要——失败主因是需求不全 + 用户没参与，需求阶段改错比维护阶段便宜 **200:1**。四类需求：业务/相关方/解决方案/过渡。相关方三层：主/次/第三。功能 = 做什么，非功能 = 质量约束 (整体)。民族志好处口诀：隐性知识/真实流程/协作/低偏差。

4. Requirements Elicitation: Documents [Lec04]

Once stakeholder requirements are analysed, prepare documents to discuss with stakeholders: **use case diagram, personas, scenarios, user stories**.

Use case diagram (as an elicitation tool)

- “A graphical representation of a relationship between an **actor** (a user, application, or system) and a solution (**use case**).” Represents the people who use the system and the tasks they perform. A use case is written as “**Verb**” +/- “**Noun**”.
- **<<include>>** = **needed actions** — “Read Book” *includes* “Open Book” (you must open the book to read it).
- **<<extend>>** = **part-of / optional actions** — “Read Book” is *extended by* “Turn Page” (turning a page is optional).

Personas

- “Fictional representations of various user types,” designed from **research and observation** of real people, to understand users’ **needs, experiences, behaviours and objectives**. Aim: clearly **differentiate stakeholders**.
- Make **2–3** personas, one page each, with a generic name/photo. Use them to ask “Would *Jim* use this?” — stops designers designing for themselves, and helps **prioritise** requirements.

Scenarios

- “Explanation of how the system can be utilised for specific tasks, presented in a structured format rather than a narrative.” They give a high-level overview and include inputs/outputs.
- **Components a scenario may include** (2024 Q1): (1) a **setting/context**; (2) **actors/users** (a persona); (3) **goals/objectives**; (4) a **plot** — the sequence of events covering the **normal flow, what can go wrong and how it is handled**, and **other parallel activities**.

User stories

- “An informal and broad description of a software feature narrated from the viewpoint of the end user” — clarifies how a feature delivers value (“As a..., I want..., so that...”).

中文助记：四种文档——用例图 / 人物角色 / 场景 / 用户故事。<<include>>= 必须 (开书才能读), <<extend>>= 可选 (翻页)。Persona 做 2-3 个，问“Jim 会用吗”防止自嗨。场景四要素：情境/参与者/目标/情节。用户故事 = 用户视角一句话。

5. Requirements Modelling: UML Diagrams [Lec05]

- “Models are utilised throughout the requirements engineering process.” A model **“is not a complete representation of the system; it purposely leaves out details and picks out the most salient characteristics”** (Sommerville).
- Four model perspectives: **External** (context/environment), **Interaction**, **Structural**, **Behavioural**.
- **UML 2.5 has 13 diagram types**. Five can represent a system: **Activity**, **Use Case**, **Sequence**, **Class**, **State** (plus **Context Models**). Auto-generating code from UML is **Model-Driven Development**.

(1) Context Models

What is / is not in the system.

- Show how the system relates to other (internal/external) systems it interacts with (authentication, finance, etc.). **“Context models define the boundaries of the system”** — they represent the key systems to be developed, the **relationship to other systems/components**, and **what NOT to develop** (to focus investment). Often part of the **non-functional** requirements.
- A use case diagram *could* have an actor, and an activity diagram *could* show flow —

but neither is “for” showing how systems inter-relate; **context models are specifically for this.**

(2) Activity Diagrams

How the system is used / a workflow.

- “Show the activities involved in a process or in data processing.” Used to **elaborate workflows** with **decision points, wait points and parallel work**. In general, **ONE activity diagram per use case** (not per use-case diagram).
- Notation: **rounded rectangles** = actions; **diamonds** = decisions; **bars** = start (split / fork) or end (join) of concurrent activities; **black circle** = start; **encircled black circle** = end.
- **Fork/Join**: a fork starts **concurrent branches** (any order); a join waits for **all** branches before continuing.

(3) Use Case Diagrams

Interactions with external actors.

- “Show the interactions between a system and its environment.” High-level “what the system does”: **actors** + use cases (ovals) + system boundary. Relationships: **association**, **<<include>>**, **<<extend>>**, **generalisation**.

(4) Sequence Diagrams

Interactions over time.

- “Show interactions between actors and the system and between system components” — good for **complex sharing of information** as a **series of messages** between components (can later specify Java method calls).
- An **alt** box (choice box) means one of the contained interactions runs; the guard is shown in **[square brackets]**. *Read each message top-to-bottom as a sentence; for*

all, describe both branches.

(5) Class Diagrams

Object classes and associations.

- “Show the object classes in the system and the associations between these classes.” Objects “represent something in the real world, such as a patient, a prescription or a doctor.”
- A class box has three sections (top to bottom): **class name / attributes (fields, optionally with types) / methods (operations)**.
- **Generalisation** (arrowhead pointing **up** to the more general class) = inheritance: “the attributes and operations linked with higher-level classes are inherited by the lower-level classes,” and subclasses may add their own. *Example:* Doctor -> Hospital Doctor / General Practitioner; Hospital Doctor -> Trainee / Registered / Consultant. (A trainee doctor is **not** a type of general practitioner — not all are related by generalisation.)

(6) State Diagrams

Reaction to events.

- “Illustrates the system’s reactions to external and internal events,” assuming a **finite set of states** with **events (stimuli)** causing transitions. Notation: **rounded rectangles** = states (may include a do action); **labelled arrows** = stimuli forcing a transition; **filled circles** = start/end.
- *Microwave example:* starts in a **Waiting** state -> press full/half power -> set time -> if door closed, **Start enabled** -> pressing Start begins **Operation** (which has substates: status check, cooking, alarm) -> on completion returns to Waiting; opening the door moves it to a **disabled** state. A **table of states and stimuli** can list the transitions.

How to choose a diagram?

1. Early requirements: **Context Models** (what is/is not in the system); **Activity Diagrams** (how the system is used).
2. Interactions: **Use Case Diagrams** (drafts of system-actor interactions), then **Sequence Diagrams**.
3. Refined / spec / design: **Class Diagrams**, **State Diagrams**.

中文助记：六种图——**Context**(边界) / **Activity**(工作流, 1 用例 1 图) / **Use Case**(做什么) / **Sequence**(时序消息) / **Class**(类与关联, 泛化 = 继承) / **State**(状态机)。模型”故意省略细节、抓最显著特征”。活动图记号：圆角矩形 = 动作、菱形 = 判定、粗条 = fork/join、实心圆 = 起止。序列图 alt = 分支选择。

6. Requirements Specification [Lec06]

- **User requirements** are for **non-technical** stakeholders — describe only the system's **external behaviour**, not architecture/design; written in natural language with simple tables/forms/diagrams.
- **Four notations for writing system specifications** (2023 Q1):
 1. **Natural language** — plain-language sentences.
 2. **Structured (natural language)** — standard **forms/templates**, tabulating requirements precisely.
 3. **Graphical notations** — UML/graphical models.
 4. **Mathematical specifications** — formal mathematical models; the most precise.

中文助记：四种记法口诀：自然语言 / 结构化 (表单模板) / 图形 (UML) / 数学 (形式化)，精确度递增。用户需求面向外行、只讲外部行为。

7. Requirements Validation [Lec07]

Why validate (three reasons, 2025 Q2a):

1. **Checking that you are right** — “the process of checking that requirements actually define the system that the customer really wants.”
2. **Avoiding rework** — “errors in a requirements document can lead to extensive rework costs. The cost of fixing a requirements problem by making a system change is usually greater than repairing design or coding errors.”
3. **Contractually agreeing** — decide exactly what to build; all stakeholders and the team **agree** (else you and the customer have very different ideas; they don’t pay until *their* idea is achieved).

Validation techniques:

1. **Requirements reviews** — “Requirements are analysed systematically by a team who check for errors and inconsistencies.”
2. **Prototyping** — “Developing an executable model of a system” and using it with end-users to verify needs and expectations.
3. **Test-case generation** — “Requirements should be testable; if a test is difficult or impossible to design, this commonly signifies that the requirements are challenging.”

中文助记：验证三理由——确认方向 (右) / 避免返工 (后期改更贵) / 契约共识。
三技术：评审 / 原型 / 测试用例生成。难点：错误越晚发现修复越贵，所以验证性价比高。

8. Prototyping [Lec08]

- A prototype clarifies requirements, explores design options, and supports iterative development.
- **Two strategies: throwaway prototyping** (built to learn/clarify, then discarded) vs **evolutionary prototyping** (refined into the final product).

- **Three purposes of a prototype:** (1) role of **technology** (e.g. company-vision demos like Apple's Knowledge Navigator); (2) **look & feel** (what UI designers do); (3) **implementation guide** (so you can tell a developer "build X").
- A prototype "helps everyone — the customer, the manager, the developer — imagine what we are building," but you **still need to tell a developer** what to build.
- **Fidelity:**
 - **Low-fidelity** — "captures the point, the functions etc.", **to help improve the ideas**; e.g. sketches / paper prototypes / wireframes; early, cheap, fast.
 - **High-fidelity** — "represents part of the reality", **to agree on the final designs** (the finalised look & feel / functionality); any stage; more expensive.
- **When to use low vs high fidelity** (2023 Q4): **low** = early, to explore/improve ideas cheaply, when you only need to test the point; **high** = to agree the final design, for realistic usability testing / a convincing demo.
- **Prototyping risks (drawbacks):** (1) investing **too much time/energy on a high-fidelity** prototype when low-fidelity would test the point; (2) **ad-hoc prototype code reused** in the real system though not built to professional standard; (3) prototyping used **instead of, rather than alongside, documentation**.

中文助记：三目的：技术/外观/实现指南。低保真= 草图, 早期便宜, 改想法；高保真= 逼真, 定终稿, 做演示。缺点：高保真花太多 / 临时代码被复用 / 拿原型代替文档。

9. Testing Methods & TDD [Lec11]

Testing methods

- **White Box Testing** — "A procedure to derive and/or select test cases based on an **analysis of the internal structure** of a component or system." (a.k.a. clear-box, code-based, logic-driven). Done at unit/integration level by developers; needs design docs/flowcharts.

- **Black Box Testing** — “A procedure to derive the test cases based on the **functionality** of the application and **not considering the internal structure**.” Higher-level (functional) testing; needs the requirements specification.
- **Black-box advantages:** tester needs **no technical background**; testing can start once development is done (testers/developers work independently); effective for large, complex apps. **Disadvantages:** without code knowledge some conditions may be missed; **complete test coverage is not possible** for large/complex projects.

White-box coverage criteria

- **Statement Coverage** — “Ensure that each code statement is executed once.”
- **Branch Coverage (Node Testing)** — “Coverage of each code branch.”
- **Compound Condition Coverage** — “For multiple conditions test each condition with multiple paths and combinations.”
- Others: **Basis Path Testing** (each independent path), **Data Flow Testing** (track each variable’s use), **Path Testing** (all possible paths), **Loop Testing**.
- White-box testing aims to ensure: **all independent paths** executed at least once; **all logical decisions** tested true *and* false; **all loops** run at their boundaries. It targets **logical, design and syntax** errors. For large systems, full path coverage is **not always possible**.

Black-box techniques

Equivalence Partitioning (group inputs by similar outcome), **Boundary Value Analysis** (values at boundaries), **Decision Table Testing**, **State Transition Testing**, **Error Guessing** (experience-based), **Comparison Testing**.

White-box vs Black-box (comparison)

	White Box	Black Box
Knowledge	knows actual code & internal structure	no knowledge of code/structure
Level	lower (unit, integration)	higher (functional)

	White Box	Black Box
Focus	the actual code, program & syntax	the functionality under test
Needs	design docs, data-flow diagrams, flowcharts	the requirement specification
Done by	developers/testers with programming knowledge	testers (no code knowledge)

Manual vs Automation testing

	Manual	Automation
Execution	by hand	with tools
Reliability	not efficient/reliable	more efficient & reliable
Accuracy	not guaranteed	guaranteed
Across OSs	difficult	easy
Best for	usability/accessibility; areas that change frequently ; new tests	frequent/repeated tests; error-prone cases; many browsers/environments

Test-Driven Development (TDD)

- **Definition (verbatim):** “Test Driven Development (TDD) is software development approach in which test cases are developed to specify and validate what the code will do. In simple terms, test cases for each functionality are created and tested first and if the test fails then the new code is written in order to pass the test and making code simple and bug-free.”
- **The Three Rules of TDD (Robert C. Martin) — learn verbatim:**
 1. “You are not allowed to write any production code unless it is to make a failing unit test pass.”
 2. “You are not allowed to write any more of a unit test than is sufficient to fail; and compilation failures are failures.”
 3. “You are not allowed to write any more production code than is sufficient to pass the one failing unit test.”

- Cycle: **Red** (write a failing unit test) -> **Green** (write just enough production code to pass) -> **Refactor**, keeping the system executing at all times.
- **TDD advantages**: integrates **specification + coding + testing** (and documentation); makes you **think about how code is used before building it**; plan before you write; on a change it **checks you haven't broken** earlier work.
- **TDD vs traditional**: TDD focuses on production code that verifies testing works; traditional focuses on test-case design. **In TDD you can achieve 100% coverage** — every line is tested.

JUnit & metrics

- **JUnit** — “A framework for writing tests for Java code” in **separate files** (keeps production code clean). General approach: write **stub** methods -> write tests calling the stubs -> **run and check they FAIL** -> write just enough code to pass.
- Key annotations: **@Test**, **@BeforeEach/@AfterEach** (run around each test), **@BeforeAll/@AfterAll** (run once), **@Disabled**.
- **Code Coverage** — “verify the extent to which the code has been executed.” **Test Coverage** — “monitor the number of tests that have been executed.”

Worked JUnit demo — monthly salary = hourly wage x 160 hours; for wage 50, expect 8000:

```
// RED: write the failing test first
@Test
void testCalc_Hourly50_Return8000() {
    SalaryCalculator sc = new SalaryCalculator();
    double monthly = sc.GetMonthlySalary(50);
    Assertions.assertEquals(8000, monthly); // fails: stub returns 0
}
// GREEN: just enough production code to pass
public double GetMonthlySalary(int i){ return i * HOURS_IN_MONTH; } // 160
// REFACTOR: improve without changing behaviour (e.g. use double param)
```

Coverage calculation (2023 Q5 — identical to the slide example), for the `if(result>0)` snippet (7 statements):

Test	Path	Statement	Condition
a=3, b=9	result>0 TRUE	5/7	1/2 (TRUE only)
a=-3, b=-9	result<=0 FALSE	6/7	1/2 (FALSE only)
Both	—	7/7 = 100%	2/2 = 100%

中文助记：白盒= 看内部结构 (开发者, 单元/集成)；黑盒= 只看功能 (测试者, 无需懂代码)。覆盖：语句/分支/复合条件。TDD 定义和三规则要背原话。循环 **Red->Green->Refactor**，先确认测试失败。JUnit 写在单独文件，@Test/@BeforeEach/@AfterEach/@BeforeAll/@AfterAll。100% 语句覆盖 $\neq$ 100% 分支/条件覆盖。

10. Integration, System & Acceptance Testing [Lec12]

Integration testing

- “Combine all of the units within a program and test them as a group”; **“Designed to find interface defects between the modules/functions”** — checks data flow between modules.
- **Types**: **Big-bang** (integrate everything then test as a whole); **Top-down** (test from top-level modules down, using **stubs** for undeveloped lower modules); **Bottom-up** (from lower modules up, using **drivers** for undeveloped higher modules); **Hybrid/Sandwich** (both directions, using stubs and drivers).
- **Test pyramid** (Google ~70/20/10): **Unit > Integration > End-to-end** tests.

System testing

- Done after unit & integration, **“once the complete system is ready and can be tested as a whole.”** Goal: check the whole system **meets the full specifications (business requirements), including non-functional ones**, and “find errors that result from **unanticipated interactions between components.**”

- **Primary goal:** convince the company the software is **good enough to give to the customer**; ends with a **sign-off** that it is ready.
- **Types of system testing:** **Usability** (how easy/user-friendly); **Load** (performance under expected real-life load); **Regression** (changes haven't broken existing features); **Recovery** (can recover from crashes); **Migration** (moves between old and new infrastructure); **Functional Completeness** (any missing functions?); **Hardware/Software** (hardware–software interactions).
- **Strategies** (you can't test every interaction comprehensively, so pick one):
 - **Performance-driven** — metrics: **response time, resource utilization, workload, throughput**; methods: **load** (increasing load to its maximum), **stress** (stability under limited hardware resources), **scalability, endurance** (expected load over a long time), **volume** (large amounts of data) testing.
 - **Requirement-driven** — “test cases, conditions and data are derived from requirements”: define completion criteria -> design cases -> execute -> verify results -> verify coverage (functional **and** non-functional) -> track defects.
 - **Scenario-driven** — use real end-to-end scenarios; **you play the scenario character**, making both intended mistakes and correct actions; one scenario tests several requirements.
- **Use-case-based testing** is an effective approach to system testing (focus on interactions).
- *Scenario-driven example (login):* check behaviour for valid email + valid password; invalid email + valid password; valid email + invalid password; both invalid; both blank; “forgot password”; valid/invalid phone + password; “keep me signed in” checked.

Acceptance testing (vs integration)

- Three differences: (1) a **separate team not involved in development** does system/acceptance testing (Dev Team vs **QA Team**); (2) the objective is to **check the system meets its specifications and is good enough for external use** (not to find integration bugs); (3) it is **validation testing, not defect testing**.

	Unit	Integration	System
What	individual pieces	combinations (subsystems)	full system
Input	unit spec	subsystem spec	full functional + non-functional spec
Output	pass/fail	bug reports	validation reports + sign-off
By who	developer	dev team	QA / testing team
Frequency	many per day	end of a sprint	prior to showing client

中文助记：集成测试找接口缺陷；4 种方式：**big-bang / top-down(stub) / bottom-up(driver) / hybrid**。系统测试 = 整系统对照全部规格 (含非功能)，目标“够好交付客户”，结束签字。验收 = **独立 QA 团队**做验证测试而非缺陷测试。金字塔 70/20/10。

11. Configuration & Deployment [\[Lec13\]](#)

Why evolution

Software constantly changes because: bugs are discovered and must be fixed; new hardware/platforms appear; competitors add features; requirements change.

Configuration Management (CM) — four activities

- **Definition (verbatim):** “Configuration Management (CM): track and control changes in the software.”
 1. **Version Control** — “keep track of the multiple versions of system components, and ensure that changes made by different developers do not interfere with each other.”
 2. **System Building** — “assemble program components, data, and libraries, then compile and link these to create an executable system.”
 3. **Change Management** — “keep track of requests for changes... working out the costs and impact, and decide if and when the changes should be implemented.”

4. **Release Management** — “Prepare software for external release and keep track of the system versions that have been released for customer use.”

Version control concepts

- **Baseline** — “an agreed description of the attributes of a product, at a point in time (snapshot)”; a change moves from one baseline to the next.
- **Centralized VCS** (e.g. Subversion) — one master repository; pros: good with binary files, full visibility, easy to learn; cons: **single point of failure**, hard for large teams.
- **Distributed VCS** (e.g. Git) — every developer clones the full history; pros: fast local actions, private/offline work; cons: slow initial checkout, more storage.
- **Branch** — “the duplication of an object under version control”; each copy can be modified separately and in parallel.
- **Five branching scenarios**: (1) **No branches** (small/medium teams from the main tree); (2) **Branch for release** (stabilise a release, then merge back — the most common); (3) **Branch for maintenance** (maintain an old build without destabilising production); (4) **Branch for feature** (develop a feature in parallel, merge back); (5) **Branch for team** (isolate sub-teams so breaking changes don’t affect others).

System building & CI

- **System building** — create a complete executable by compiling/linking components, libraries, config; uses three platforms: **development system**, **build server**, **target environment**.
- **Three build platforms**: **development system** (compilers/editors; devs check code out of the VCS into a private workspace); **build server** (builds the definitive executable, may use external libraries not in the VCS); **target environment** (where the system runs).
- **Build-system functionality**: build-script generation, VCS integration, minimal re-compilation, executable creation, test automation, reporting, documentation generation.

- **Continuous Integration (CI)** — “Practice of automating the integration of code changes from multiple contributors into a single software project” (Booch, 1991). Devs frequently merge into a central repo; each check-in triggers an automated build + tests.
- **CI workflow:** check out the mainline -> build & run tests (if broken, tell whoever last checked in) -> make your changes -> build & test in your private workspace -> check into the build system (don’t commit as baseline yet) -> build on the **build server** and test (in case others changed components) -> only if it passes, **commit as a new baseline**.
- **CI pros:** integration problems found early; the mainline is always the definitive working system. **Cons:** slow to build/test for very large systems; can’t run tests locally if the development platform differs from the target platform.

Change management

- “Ensure that system evolution is a managed process and that priority is given to the most **urgent and cost-effective** changes.” Change analysis weighs: consequences of *not* changing, benefits, number of users affected, costs, release cycle.
- **Types of change:** **Fault repairs** (cheap), **Environmental adaptation** (cheap), **Functionality addition** (expensive).

中文助记：CM=追踪并控制软件变化，四活动：版本控制 / 系统构建 / 变更管理 / 发布管理。Baseline= 约定的快照。集中式 (SVN, 单点故障) vs 分布式 (Git, 本地全历史)。CI= 自动集成, 问题早发现, 主线永远可用。变更优先紧急且划算的；改动三类：修错 (廉)/适配 (廉)/加功能 (贵)。

12. Evolution & Maintenance [Lec14]

- “Maintenance involves: correcting errors not discovered earlier; improving the implementation of system units; enhancing services as new requirements are discovered.”

“Normally, this is the longest lifecycle phase.”

- **Change is inevitable:** new requirements emerge in use, the business environment changes, errors must be repaired, new equipment is added, performance/reliability must improve. **85–90% of the software budget** goes to **changing/evolving** existing software, not new development (rebuilding is risky, slower, costlier).

Types of maintenance (verbatim)

1. **Maintenance to repair software faults** — “changing a system to correct faults.”
2. **Maintenance to adapt software to a different operating environment** — “changing a system so that it operates in a different environment (computer, OS, etc.).”
3. **Maintenance to add to or modify the system’s functionality** — “modifying the system to satisfy new requirements.”

Maintenance costs & prediction

- Maintenance costs are usually **greater than development costs (2x–100x)**. Affected by: **team stability, contractual responsibility, staff skills, program age and structure.**
- **Maintainability prediction** uses metrics: number of corrective-maintenance requests, average time for impact analysis, average time to implement a change, number of outstanding change requests.

Lehman’s laws of program evolution (Lehman & Belady)

Law	Description
Continuing change	a program used in a real-world environment must change or become progressively less useful
Increasing complexity	as it changes, structure becomes more complex; extra resources needed to preserve/simplify it
Large program evolution	self-regulating; size, release interval and reported errors stay ~invariant per release
Organisational stability	development rate is ~constant, independent of resources

Law	Description
Conservation of familiarity	incremental change per release is ~constant
Continuing growth	functionality must continually increase to keep user satisfaction
Declining quality	quality declines unless modified to reflect environment changes
Feedback system	evolution is a multi-agent, multi-loop feedback system

Reengineering & refactoring

- **Software Reengineering** — “Restructure or rewrite part or all of a legacy system **without changing its functionality**” (redocument, refactor architecture, translate language, reengineer data). Benefits: **reduce risk, reduce cost**.
- **Reengineering process**: source-code translation -> reverse engineering (understand the program) -> program-structure improvement -> modularisation -> data reengineering.
- **Refactoring** — “Make improvements to a program to **slow down degradation** through change”; **preventative maintenance** that improves structure/complexity/readability, **not** adding functionality; a continuous process throughout development.
- **Bad smells of code** (signals to refactor): **duplicate code** (extract into one method); **long methods** (split into shorter ones); **switch/case statements** (often duplication — use **polymorphism/inheritance**); **data clumping** (the same group of data items recurs — wrap in an object); **speculative generality** (unused generality).

中文助记：维护是最长阶段，预算 85-90% 花在演化旧软件。维护三类 (背原话)：修故障 / 适配环境 / 增改功能。Lehman 核心矛盾：持续变化 <-> 复杂度递增/质量下降。再工程 = 不改功能重构旧系统；重构 = 预防性维护、改结构不加功能；坏味道：重复代码/长方法/switch/数据团/过度设计。

13. Software Quality [Lec15]

- **Software Quality means doing every aspect well.** “Most people presume Software Quality is a stage (Release/Acceptance testing) — **it’s not!**” If quality management is **pervasive and a culture**, the software won’t fail acceptance testing as often.
- **“All SE Processes are designed to improve software quality.”** To do acceptance testing well you need good Reqs & Specs; to do unit testing well you need clear specs; to code well you need coding standards.

QA team — three (four) things

1. **Planning for quality** — a **Project Quality Plan** “defines what ‘high-quality’ software actually means for a particular project” (sections: product intro, product plans, **process description**, **quality goals**, risks).
2. **Defining standards** — **Product Standards** (documentation, coding conventions, class structure) and **Process Standards** (when reviews/testing happen). **ISO** helps: **ISO 9000/9001** (quality management), **ISO/IEC 12207** (life-cycle processes), **ISO 9241** (accessibility), **IEC 61508** (safety).
3. **Checking for quality** — (a) identifying points for **inspections**; (b) identifying relevant **measures**. The QA team should be **separate from dev teams** and report above the project manager.

Reviews & inspections

- **Inspections** find lots of bugs: Fagan (1986) estimated **60%+**; Prowell (1999) claimed **90%+** of defects.
- **Traditional inspection**: 3–7 people, ~2 hours, focus on **finding problems and non-conformance** (not solving them); produce documentation as evidence of quality.
- **Roles**: **Review Leader** (chairs), **Recorder** (documents issues), **Reader** (reads through the code/design), **Reviewer(s)** (identify problems). **Fagan inspections** (IBM, code-specific): Moderator, Designer, Coder/Implementer, Tester.

- **Common inspection problems** (and fixes): criticising the **person** not the document (moderator must prevent this); people **fear being judged** (avoid inviting line managers); people **don't prepare** (ensure they read it first); people **discuss every problem as found** so little gets inspected.
- **Quality-driven culture**: “there is much more to quality management than standards and bureaucracy.” The risk is QA becoming “the nasty people”; better to let teams own what *excellence* means and have QA **facilitate** (help teams be fast), e.g. a launch-approval tool per product.
- **Agile**: pair programming is an agile code inspection; **retrospectives** (after each sprint, team reflects) vs **postmortems**.

中文助记：质量 = 把每个方面都做好，是文化/过程不是阶段；所有 SE 流程都为提质量。QA 三 (四) 件事：规划质量 (**Quality Plan**) / 定标准 (产品 + 过程, **ISO**) / 查质量 (**inspection** 点 + 度量)，QA 团队应独立于开发。检查重在找问题不解决；角色：Leader/Recorder/Reader/Reviewer；Fagan 专查代码。

14. Agile Methods [Lec16]

History & motivation

- Traditional **Waterfall**: “software is developed in distinct phases, each leading to the next phase in a sequence resembling a waterfall. **Once a phase is complete, the process moves on to the next phase and there is no turning back.**”
- 1990s concerns with waterfall: heavy top-down, expensive/slow, depends on lots of documentation, long waits between teams, **hard to manage change, difficult to measure progress within each phase**. The **Agile Alliance (2001)** said methodologies should be **adaptable to the situation** and avoid unnecessary bureaucracy.

Agile principles (the 12)

1. Highest priority: **satisfy the customer through early and continuous delivery.**

2. **Welcome changing requirements**, even late in development.
3. **Deliver working software frequently** (weeks, not months).
4. Business people and developers **work together daily**.
5. Build projects around **motivated individuals**; trust them.
6. **Face-to-face conversation** is the most efficient communication.
7. **Working software is the primary measure of progress**.
8. **Sustainable development** — a constant pace indefinitely.
9. **Technical excellence and good design** enhance agility.
10. **Simplicity** — maximising the work *not* done.
11. Best architectures/requirements emerge from **self-organising teams**.
12. The team **regularly reflects and tunes** its behaviour.

Frameworks (all uphold the Agile principles)

- **Scrum** — “a lightweight framework that helps people, teams and organizations generate value through adaptive solutions”; **about optimising time & delivery**. Roles: **Scrum Master** (keeps the process, removes obstacles), **Product Owner** (manages the **Product Backlog**), **Team**. Artifacts: **Product / Sprint Backlog**; work in time-boxed **sprints**.
- **Kanban** — **about speed of delivery** (visualise workflow, limit work-in-progress).
- **XP (eXtreme Programming)** — **about team effectiveness**; “a fast ‘extreme’ approach to iterative development” — new versions may be built daily, **increments delivered to customers every 2 weeks, all tests applied before a build is accepted** and **builds only released when they pass all tests**.
- **How XP meets the Agile values**: customer collaboration = **full-time customer engagement**; working software = **small, frequent releases**; individuals & interactions = **pair programming, collective ownership**, no long hours; responding to change = **regular system releases**.

How to apply Agile to each stage

- **Agile requirements** — use **user stories**, not big requirement documents.
- **Agile design** — use **prototypes**, not heavy specification documents.
- **Agile implementation** — **test-driven, paired development**.
- Choosing **Agile vs Plan-driven** comes down to two questions: (1) is a very detailed spec/design needed *before* implementation? (2) is incremental delivery with rapid customer feedback realistic? Agile suits **small/medium projects with an involved customer**.

Scrum roles, artifacts & events

- **Roles:** **Scrum Master** (keeper of the process — makes it run smoothly, removes obstacles, facilitates meetings); **Product Owner** (sole owner of the **Product Backlog** — expresses items clearly, orders them by value, keeps it visible); **Team** (analysts, designers, developers, testers).
- **Artifacts:** **Product Backlog** (ordered list of everything the product may need); **Sprint Backlog** (items selected for the current sprint + a plan); **Increment** (the working software produced).
- **Events:** **Sprint** (a time-box, e.g. 2–4 weeks); **Sprint Planning** (pick backlog items, set the sprint goal); **Daily Scrum / stand-up**; **Sprint Review** (demo the increment); **Sprint Retrospective** (reflect and improve).
- **Burndown charts** track remaining work across the sprint.

Obstacles / cons of Agile

- Teams tend to **neglect documentation**; weak initial design forces **frequent refactoring**; **harder to practise** (members must know Agile well); time-boxing means some features **miss the timeline** (extra sprints, cost); relies on an **involved customer** (hard long-term); important tasks can be forgotten; organisational change is slow.

中文助记：瀑布 = 顺序、阶段完成不回头；缺点：抗变、阶段内进度难测。敏捷宣言重点：早交付/拥抱变化/常交付可用软件/面对面/可用软件是进度标准/简单。框架：Scrum(时间 & 交付,SM/PO/Team,sprint+backlog) / Kanban(速度) / XP(团队,结对 +2 周交付 + 全测通过才接受)。障碍：轻文档/频繁重构/难实践/难按期/依赖客户。

15. Risk Management [Lec17]

Definitions (verbatim — learn exactly)

- “A risk is the probability of unwanted consequences of an event and decision.”
- “An opportunity is the probability of exceeding expectations. A risk is the probability of failing to meet expectations.”
- “A risk is not a problem. A risk is a potential problem over which we have some choices.”
- **Causes of risk**: uncertainty in **time**, in **control**, in **information**. “There is no software project without risks.”

Identifying risks

- **Interviewing/Brainstorming, Voluntary Reporting** (reward whoever flags a risk), **Decomposition** (every “TBD” is a potential risk), **Critical Path Analysis, Assumption Analysis, Risk Taxonomies** (checklists from past projects).
- **Shall-not requirements**: actively think about what should **not** happen (e.g. people accessing others’ personal data; services going down).

Product risk reduction (3, by design)

1. **Hazard Avoidance** — so it cannot occur.
2. **Hazard Detection & Removal** — so systems recover nicely.
3. **Damage Limitation** — so the impact is limited.

Project risk: prioritisation & strategies

- **Prioritise** by **probability** — very low (<10%) / low (10–25%) / moderate (25–50%) / high (50–75%) / very high (>75%) — **and effect** — catastrophic (project fails) / serious (expense, delays) / tolerable (in contingency) / insignificant (don't care). **Both matter.**
- A **Risk Review Board (RRB)**, led by the project manager with representatives from each area, documents each risk's **type** (cost/schedule/technical), **severity** and **management plan**.
- **Risk-control strategies** (verbatim):
 - **Avoidance strategies** — “actions taken to reduce the risk happening.”
 - **Minimization strategies** — “reducing the impact if it happens.”
 - **Contingency plans** — “what you will do/change if it happens.”
 - (**Transfer** — pass risk to a third party, e.g. a subcontractor.)
 - **“Avoidance is always the best strategy, if possible”** — take pre-emptive action.
- **Risk monitoring** — keep checking whether any risk is starting to happen.
- **Risk analysis at 3 stages**: **Preliminary** (critical requirements), **Lifecycle** (implementation decisions), **Operational** (UI/interaction decisions).

中文助记：背原话——风险 = 不良后果/未达预期的概率；机会 = 超预期的概率；风险 ≠ 问题 (是仍有选择权的潜在问题)。识别法：访谈/自愿上报/分解 (TBD)/关键路径/假设分析/风险清单。产品降险：避免发生/检测移除/限制损害。项目策略：**Avoidance**(降发生) / **Minimisation**(降影响) / **Contingency**(发生后) / **Transfer**，规避最佳。按概率 × 影响排序。对应 2025 Q5：离职-> 最小化、接口延期-> 应急、客户改 UI-> 规避。

16. Project Planning [Lec18]

Project plan & scheduling

- A project plan covers: **Introduction, Project Organisation, Risk Analysis, Hardware & Software, Work Breakdown, Project Schedule, Monitoring & Revision Plans.**
- **Milestone** — “the end point of an activity.” **Deliverables** — “results delivered to customers.” Every task should produce a **tangible output**; tasks should be **1 to 8–10 weeks** (break down anything longer).

Planning diagrams

- **PERT (1958)** — tasks and dependencies (good for **dependencies and parallel tasks**). **Critical Path Method (1960)** — risk analysis. **Gantt chart (1910)** — adds time to tasks/dependencies. **Staff-allocation charts** — who does what.

PERT & critical path (2025 Q3)

- Method: “Take a PERT chart; identify all the paths; identify the length of time for each path; **the longest path (worst case) is the critical path**” — it estimates the **worst-case** effort.
- “**The Critical Path is the Bottleneck Route**” — modifying a task on the critical path changes the whole project duration; its activities have **zero slack/float**.

Gantt charts

- A **Gantt chart** adds **time** to tasks/dependencies. A good one shows: task overviews (with IDs), **categorised work, milestones and dates**, and **parallel work** drawn on a timeline. (Use **staff-allocation charts** to show who does each task.)

Reviewing progress & Agile planning

- **Milestones are times to review progress** and the plan; a plan is an estimate, so check for **slippage**. If small, it may be recovered later; if **serious**, fall back on your risk strategies and **re-plan** — change tasks, negotiate for more money, or decide

whether the project should **continue or be cancelled**.

- **Cost estimation** — two approaches: **experience-based** (from similar past projects / a manager's familiarity) and **algorithmic cost modelling** (mathematical, from estimated effort). A common **rule of thumb** is to add a **30–50% contingency** for what the plan missed or for risks that occur.
- **Two ways to budget**: by **work** (estimate the developer time it needs) or by **developer time** (define how much work fits) — the latter is why **Scrum** is popular: you agree a number of **sprints** for a team and “expanding a project is buying more sprints”.

中文助记：里程碑 = 活动终点，交付物 = 给客户的成果；任务 1–10 周。图：**PERT(依赖/并行)** / **CPM(关键路径)** / **Gantt(加时间)** / **人力分配**。关键路径 = 所有路径里最长那条(瓶颈, 零时差, 决定工期)。进度按里程碑复查 **slippage**，严重则用风险策略重新计划。估算：经验法/算法模型，加 30–50% 余量；敏捷 = 买 **sprint**。

17. Likely Exam Questions (by topic)

Past papers (2023–2025) repeatedly test the items below. For each, know the **verbatim definition** plus a few bullet points.

Requirements (Lec03–07)

- “Outline *FOUR* components that a scenario might include.” -> setting / actors / goals / plot (normal flow, what can go wrong, parallel activities). [Lec04]
- “Outline *FOUR* benefits of ethnography.” -> tacit knowledge / real process / cooperation / low bias. [Lec03]
- “Describe *FOUR* notations for writing system specifications.” -> natural / structured / graphical / mathematical. [Lec06]
- “Why is validating requirements important? Give reasons + techniques.” -> checking you're right / avoid rework / contractual agreement; reviews / prototyping / test-case

generation. [Lec07]

- “Draw a use case diagram” / “describe a sequence diagram in N sentences” / “which action sequences are permissible in this activity diagram?” [Lec05]
- “When to use low- vs high-fidelity prototypes?” + one drawback. [Lec08]

Development & testing (Lec11–13)

- “List the THREE rules of TDD.” -> verbatim (failing test drives code; test just enough to fail; code just enough to pass). [Lec11]
- “Define statement coverage; compute it” for given inputs (5/7, 6/7 -> 100%). [Lec11]
- “What testing type is this, and its benefits?” (feature checks = functional/black-box; no code knowledge, user viewpoint). [Lec11]
- “Compare and contrast FOUR testing stages.” -> unit / integration / system / acceptance (scope, who, focus grow). [Lec12]
- “Name the integration approaches” -> big-bang / top-down (stubs) / bottom-up (drivers) / hybrid. [Lec12]
- “What are the four configuration-management activities?” [Lec13]

Management (Lec14–18)

- “Difference between Waterfall and Agile.” -> sequential & no turning back vs iterative & embraces change. [Lec16]
- “Choose two Agile frameworks and explain.” -> Scrum / Kanban / XP. [Lec16]
- “What are the obstacles to Agile?” -> neglect docs / refactoring / hard to practise / miss timeline / needs involved customer. [Lec16]
- “Difference between a risk and an opportunity; is a risk a problem?” -> verbatim definitions; risk $\neq$ problem. [Lec17]
- “What is the risk-management workflow?” -> identify -> analyse -> plan -> monitor. “Strategies, and which is best?” -> avoidance / minimisation / contingency; avoidance. [Lec17]
- “Draw a PERT chart; identify the critical path and explain why it matters.” [Lec18]

- “Types of maintenance / Lehman’s laws.” [Lec14]

18. Worked Examples

A. Use case diagram: library system [Lec05]

Actors: **Librarian, Student, Staff** (Student/Staff can generalise to *Member* for borrow/return). Use cases by actor:

Actor	Use cases
Librarian	Input/Modify/Remove record; Borrow; Return
Student	Borrow; Return; Pay fine; Leave review; Edit profile
Staff	Borrow; Return; Borrow journal; Reserve up to 3 books; Edit profile

Show **<<extend>>**: *Pay fine* extends *Return book* (only when overdue).

B. Activity diagram: fork / join [Lec05]

Order processing: Receive Order -> **fork** into concurrent Authorize Payment (-> decision: *failed* -> Cancel Order; *succeeded* -> join) and Assign Item to Order -> **join** -> Dispatch Order. Dispatch needs **both** branches + payment success; Cancel only follows a failed payment. **Permissible sequences: d, e, h, i** (concurrent branches may run in either order).

C. Statement & condition coverage [Lec11]

For `if(result>0)` (7 statements): `a=3, b=9` -> statement **5/7**, condition 1/2; `a=-3, b=-9` -> statement **6/7**, condition 1/2; **both together -> statement 7/7 = 100%, condition 2/2 = 100%**. Minimum test cases for 100% statement coverage of an `if/elif/else` (n branches) = **n cases**.

D. PERT critical path [Lec18]

Tasks A(6) B(3,A) C(10,B) D(6,C) E(4,C) F(4,E) G(3,D&F) H(8,C). Paths: A-B-C-D-G = **28**; A-B-C-E-F-G = **30**; A-B-C-H = **27**. **Critical path = A-B-C-E-F-G (30)**, zero slack.

19. Exam-Day Quick-Recall Sheet

Topic	Must remember (English)	Lec
SE	systematic approach to dev/operation/maintenance;	01
Git areas	software needs “a big process” working dir -> staging (index) -> repository	02
Failure cause	incomplete requirements + no user involvement; 200:1 cost saving early	03
Requirement types	business / stakeholder / solution / transition	03
Stakeholders	primary / secondary / tertiary	03
Functional vs NF	services it provides vs constraints on the whole system	03
Scenario components	setting / actors / goals / plot	04
<>/<>	include = needed; extend = optional	04
6 UML models	context / activity / use case / sequence / class / state	05
Activity notation	rounded rect=action, diamond=decision, bar=fork/join	05
4 spec notations	natural / structured / graphical / mathematical	06
Validation reasons	checking you’re right / avoid rework / contractual agreement	07
Validation techniques	reviews / prototyping / test-case generation	07
Fidelity	low = improve ideas (cheap) / high = agree final design	08
White vs Black box	internal structure vs functionality	11
Coverage	statement / branch / compound condition	11

Topic	Must remember (English)	Lec
TDD 3 rules	failing test first; test just enough to fail; code just enough to pass	11
Integration types	big-bang / top-down (stubs) / bottom-up (drivers) / hybrid	12
Testing stages	unit -> integration -> system -> acceptance	12
CM 4 activities	version control / system building / change mgmt / release mgmt	13
CI	automate integration of changes into one project	13
Maintenance types	repair faults / adapt environment / add-modify functionality	14
Lehman	continuing change + increasing complexity + declining quality	14
Quality	doing every aspect well; a culture, not a stage	15
QA 3 things	planning / defining standards / checking	15
Inspection roles	leader / recorder / reader / reviewer (Fagan for code)	15
Waterfall vs Agile	sequential, no turning back vs iterative, embraces change	16
Frameworks	Scrum (time) / Kanban (speed) / XP (team)	16
Risk	prob. of failing expectations; risk $\neq$ problem	17
Risk strategies	avoidance / minimisation / contingency (+transfer); avoidance best	17
Product risk reduction	hazard avoidance / detection & removal / damage limitation	17
Critical path	longest path; bottleneck; zero slack; sets duration	18
Estimation	experience-based / algorithmic; +30–50% contingency	18

Answering tips (Revision deck): write in English, **draw diagrams when asked** (label every actor, node, dependency and guard), answer the multiple-choice, and **write as much as you can**.

20. Glossary of Key Terms

Term	One-line meaning	Lec
Software process	the activities (and order) that produce a software product	01
Repository	store of all files + full commit history	02
Branch / merge	parallel copy of work / combining branches (may conflict)	02/09
Functional requirement	a service the system should provide	03
Non-functional requirement	a constraint on the system as a whole	03
Stakeholder	anyone with an interest in the system	03
Persona	(primary/secondary/tertiary) fictional representation of a user type	04
Scenario	structured account of how a user achieves a goal	04
User story	informal feature description from the user's viewpoint	04
Use case	a unit of system functionality used by an actor	04/05
<<include>> / <<extend>>	needed sub-use-case / optional sub-use-case	04
Context model	defines system boundaries and related systems	05
Activity diagram	workflow of one use case (fork/join, decisions)	05
Sequence diagram	time-ordered messages between objects (alt = choice)	05
Class diagram	classes + attributes + methods + generalisation	05
State diagram	states + stimuli (transitions)	05
Validation	checking requirements define what the customer really wants	07
Prototype	early executable model; throwaway vs evolutionary	08
Fidelity	low (improve ideas) vs high (agree final design)	08
White-box testing	test cases from internal code structure	11
Black-box testing	test cases from functionality (no code knowledge)	11

Term	One-line meaning	Lec
Statement coverage	executed statements / total statements	11
TDD	tests written first to specify & validate the code	11
Stub / driver	placeholder for lower / higher modules in integration	12
System testing	whole system vs full spec (incl. non-functional)	12
Acceptance testing	customer validates the system; QA team; sign-off	12
Configuration management	track & control changes (VCS/build/change/release)	13
Baseline	an agreed snapshot of the product at a point in time	13
Continuous Integration	automate integrating changes into one project	13
Maintenance	corrective / adaptive / perfective changes after release	14
Reengineering	restructure a legacy system without changing functionality	14
Refactoring	improve structure to slow degradation; no new functionality	14
Software quality	doing every aspect well; a pervasive culture	15
Inspection	team reviews an output to find (not solve) problems	15
Waterfall	sequential phases, no turning back	16
Agile	iterative, incremental, embraces change	16
Sprint / backlog	time-box of work / ordered list of items	16
Risk	probability of failing to meet expectations (not a problem)	17
Avoidance / minimisation / contingency	reduce chance / reduce impact / plan if it happens	17
Milestone / deliverable	end of an activity / result given to the customer	18
Critical path	longest path through a PERT chart; zero slack	18

中文助记：这张术语表把全课要背的英文名词一次性收齐——考前快速扫一遍，确认每个术语都能用英文说出一句定义。